What Survives an SBOM Conversion? A Source-Grounded PURL Preservation Audit

Denise M. Case
School of Computer Science and Information Systems
Northwest Missouri State University
Maryville, Missouri, USA

***Abstract*—Software bills of materials (SBOMs) move between formats and tools, but a valid output document need not preserve the identifier information needed by downstream security analysis. We investigate a narrow, auditable commitment: whether a Package URL (PURL) present in a source component survives an SPDX or CycloneDX transformation in the target's canonical semantic location. We specify the obligation from published standards, freeze an evaluator before engineering validation, and distinguish actual preservation failures from unsupported, unanchorable, and underdetermined cases. On a preserved historical SPDX input, the unchanged evaluator reproduces a documented CycloneDX CLI PURL-relocation regression and its subsequent correction across released tool versions. A separately constructed ten-source corpus and thirty-route matrix provide a prospective generalization study, not yet executed. The contribution is an inspectable protocol for asking what a representation transformation preserves, with explicit limits on what its verdicts establish.**

***Index Terms*—software bill of materials, SBOM, Package URL, PURL, CycloneDX, SPDX, software supply chain, preservation conformance**

I. Problem and Research Question

An SBOM converter can produce a syntactically valid target document while moving, dropping, or altering information relied on by downstream tools. In a documented conversion of the `tzdata` package, a source SPDX PURL appears in a CycloneDX property but not in the canonical `component.purl` field [4]. A consumer reading only that canonical field would not see the identifier, although the converted document retains it elsewhere. This is a plausible interoperability consequence, not evidence of a missed vulnerability match in an actual scanner. A converter can satisfy an output schema without meeting all downstream identifier-location assumptions. Such a discrepancy can complicate package correlation or vulnerability matching without demonstrating that an attack occurred. SPDX and CycloneDX describe different representations of package identification, and PURL specifies the identifier syntax [1]–[3].

Our question is deliberately smaller than whether two descriptions identify the same real-world software artifact: *Can a preservation obligation, defined from authoritative sources before examining a transformation result, detect when identity-bearing information is preserved, altered, dropped, or relocated?* We test information in a declared representation role; we do not infer referent identity, authenticity, vulnerability, or security from a matching identifier.

II. Source-Grounded Preservation Audit

The pilot freezes a typed representation-preservation commitment for PURL. A format adapter extracts identifier-bearing fields according to each source or target standard. A separate evaluator checks the commitment after associating a source component with a target component under a stated anchoring procedure. The target must retain the required PURL in its canonical location; retaining it in an unrelated property does not satisfy this obligation.

Evaluation returns evidence and an explicit classification, including `PRESERVED` or a specified `VIOLATED_*` outcome when the obligation can be decided. Unsupported transformations, execution failures, non-applicability, `UNANCHORABLE` component correspondence, and `UNDERDETERMINED` identifier choice are tracked separately. In particular, a failed conversion cannot be counted as an identifier-preservation failure, and an unanchorable case cannot silently disappear from coverage statistics. A source-target pair is assessed only under the declared anchoring and applicability rules; arbitrary document similarity is not treated as component correspondence.

The scientific control is a two-stage freeze. Freeze 01 fixes the commitment, interpretation rules, evaluator, and adjudication procedure before the historical engineering-validation exercise. Freeze 02 must separately fix corpus membership, source bytes and hashes, converter/runtime versions, commands, and supported routes before any held-out outputs are inspected. Defects in frozen scientific content require a recorded freeze break and new freeze, not silent adaptation to observed results. In contrast to a general document diff, the method exposes a particular testable obligation and a finite explanation for either a verdict or a refusal to decide.

III. Completed Engineering Validation

We evaluated a preserved SPDX source associated with the reported `cyclonedx-cli` PURL conversion defect [4]. Under the same frozen commitment and evaluator, version 0.31.0 yields `VIOLATED_RELOCATED`, whereas version 0.32.0 yields `PRESERVED`. An eight-release sweep further distinguishes a source parse failure in 0.29.0, relocation verdicts from 0.29.1 through 0.31.0, and preservation verdicts from 0.32.0 through 0.33.1. The sweep preserves execution outcomes separately from conformance judgments and checks observations against predeclared expectations.

These are *engineering-validation* observations, not blind discovery: the historical defect shape was known during apparatus development. The result establishes that the frozen evaluator can discriminate a known representation defect from its absence across this tested release boundary; it does not estimate how frequently such defects occur in other converters. Its value here is methodological: the known-bad and known-good cases provide positive controls for interpreting the evaluator before it encounters any prospective corpus outputs.

IV. Prospective Generalization Study

A pre-outcome sampling procedure screens 110 JSON examples from fixed revisions of the official CycloneDX and SPDX example repositories. Of 21 eligible candidates, ten source SBOMs (eight CycloneDX and two SPDX) were selected and preserved. A matrix of ten sources by three converters (Anchore Syft, Protobom `sbom-convert`, and SPDX `cdx2spdx`) contains 30 declared routes: 20 planned and ten classified as unsupported before execution. These unsupported routes are not counted as failures. Sixteen planned routes run CycloneDX to SPDX; four run SPDX to CycloneDX, the direction of the validated defect. Source selection excludes transformation outputs and evaluator verdicts; hashes, converter artifacts, commands, and target-validation steps are recorded.

The complete pre-Freeze-02 verification gate has passed, but Freeze 02 has not yet been established and no held-out transformation output has been examined. The prospective evaluation will report both conformance verdicts and their coverage: supported and successfully executed routes, independently anchored components, underdetermined cases, and confirmed versus rejected violation candidates. Findings will be adjudicated against the frozen obligation and source specifications; known-defect rediscovery will be distinguished from previously unrecorded failures. The principal outcome is not merely a count of detected mismatches. It is a traceable set of source-target observations, validity and anchoring decisions, preservation classifications, and reasons for cases where a security-relevant conclusion cannot be drawn. This enables the workshop to evaluate both detection and the boundaries of detection.

V. Interpretation and Workshop Discussion

The proposed contribution is not a new SBOM schema, a general proof of component identity, or an assertion that one converter is secure. It is a reproducible way to state *which* identity-bearing information a transformation must preserve, *where* it must appear, and *when* available evidence does not justify a yes/no answer. This is motivated by broader work on explicit identity commitments [5], but the present experiment is a preservation-conformance test, *not* an operational-identity partition audit.

We invite discussion of three design choices: Should canonical-field preservation be a separately testable SBOM conformance obligation? What component-anchoring evidence is sufficient to interpret converter outcomes? And how should tools report preservation coverage when unsupported or unanchorable cases are substantial? These questions suggest a useful interoperability contract: producers declare which identifier-bearing fields they preserve, and consumers can verify those declarations against retained evidence without assuming semantic equivalence of all fields.

The initial corpus is intentionally small and drawn from standards-project examples. It is not representative of production SBOM traffic. Any findings remain conditional on that sampling frame, the pinned toolchain, the evaluator, and the tested platform; extending the method to richer semantic identities requires new obligations and independent evidence. No newly discovered vulnerability or attack is described here; if the prospective study uncovers one, affected maintainers will be notified before submission of a report describing it. The pilot protocol and supporting records are available in the public study repository [6].

References

[1] Ecma International, "ECMA-427: Package-URL (PURL) Specification," 1st ed., Dec. 2025. [Online]. Available: https://ecma-international.org/publications-and-standards/standards/ecma-427/

[2] Ecma International, "ECMA-424: CycloneDX Bill of Materials Specification," 2nd ed., Dec. 2025. [Online]. Available: https://tc54.org/ecma424/

[3] SPDX Project, "SPDX Specification, Version 2.3.1," Clause 7 and Annex F. [Online]. Available: https://spdx.github.io/spdx-spec/v2.3.1/

[4] CycloneDX, "SPDX to CycloneDX: no purl in converted component," `cyclonedx-cli`, issue 424. [Online]. Available: https://github.com/CycloneDX/cyclonedx-cli/issues/424

[5] D. M. Case, "Operational Identity: A Finite Audit of Declared and Implemented Rules of Sameness," arXiv:2607.20729, 2026.

[6] D. M. Case, "Identity Preservation in Software Supply Chains: Pilot repository," 2026. [Online]. Available: https://github.com/structural-explainability/se-pilot-identity-preservation-supply-chain-software